

목차

PORTFOLIO · 김단비 · FULLSTACK

01

COVER

커버 — 풀스택 Portfolio 메인

02

01 · EXPERIENCE

실무 경력 — 2년의 운영 오너십

03

02 · PROJECT

프로젝트 #1 — 다2조부르릉

04

03 · PROJECT

프로젝트 #2 — ants-camp

05

04 · CAPABILITY

AI 워크플로우 — 하네스 엔지니어링

서버에서 시작해 화면까지 잇는 기능의 시작부터 끝까지 책임지 는 풀스택.

회원제 SaaS 운영팀에서 뷰어 UX·동시성 제어·RDBMS 마이그레이션을 화면과 서버 양쪽에서 직접 풀었고,
최근 프로젝트에서는 RAG 백엔드부터 React 운영 콘솔까지 한 기능을 끝까지 구현했습니다.

사용자가 체감하는 문제를 시스템 구조로 해결하고, "왜 이 기술을 쓰는가"를 끝까지 묻습니다.

github.com/danbeekimm · velog.io/@danbeekimm · ants-camp 라이브 데모 · 다2조부르릉 라이브 데모

WORKS

프로젝트 · 경력

세 작업을 시간순으로. 화면과 서버 양쪽에서 사용자 문제를 풀어 온 궤적 — 뷰어 UX·동시성을 실무에서 풀고, 부트캠프에서 RAG 백엔드부터 React 운영 콘솔까지 한 기능을 끝까지 구현했습니다.

EXPERIENCE · 실무 경력

2022.11 ~ 2025.03

풀스택 개발자 — 회원제 특허 SaaS의 고도화·운영

대량 문서 뷰어의 도면·텍스트 실시간 동기화를 부분 렌더링 → 비동기 갱신으로 재설계해 깜빡임·포커스 강탈을 제거하고, 서버에서는 분할 비동기 다운로드의 이용량 우회 race condition을 "임계구역 위치 재배치"로, PostgreSQL → MSSQL 전면 마이그레이션의 슬로우 쿼리를 옵티마이저 단위로 풀었습니다.
화면에서 느낀 한계가 서버 설계로, 다시 부트캠프 학습으로 이어졌습니다.

만족도 2위

뷰어 동기화 UX 개선 · 연간 조사

원천 차단

다중 창 동시 요청 한도 우회

동일 수준

벤더 변경 후 응답 성능 유지

JSP · AngularJS · jQuery · 비동기 렌더링 · 스크롤 동기화 · Spring · MyBatis · MSSQL · T-SQL UDF · 비동기 동시성

담당: 6개 검색 사이트 유지보수·개선 — 대량 문서 뷰어 · 다운로드 · 비교보기

상세 보기→

PROJECT #1 · 다2조부르릉

6인

정합성 보장과 장애 격리를 설계한 B2B 물류 MSA

12개 마이크로서비스 · DDD 4계층 해사고날 · "검증=동기, 통지=비동기" 통신 분리

직전 모듈리식의 한계(한 도메인 결합이 전체 멈춤)를 계기로 MSA + DDD로 확장.
분산 트랜잭션 없이 Outbox 패턴으로 이벤트 정합성을 보장하고, OR-Tools VRPTW로 B2B 시간창 SLA를 제약 조건으로 모델링했습니다.
비관적 락은 스터디에서 학습한 트레이드오프를 실제 적용한 사례.

at-least-once

Outbox · 결과적 일관성

5분 → 3초

VRPTW 자동 배정 (100건 · 데모)

DB-level

비관적 락으로 race 차단

Spring Cloud 2025 · Kafka · Outbox · OpenFeign · Resilience4j CB · OR-Tools VRPTW · QueryDSL · PESSIMISTIC_WRITE

담당: company-service 전담 + DeliveryManager 도메인

라이브 데모 · da2joburureung

상세 보기→

PROJECT #2 · ants-camp

5인

RAG 백엔드부터 React 운영 콘솔까지 끝까지 구현

모의투자 MSA · LLM 정량 평가 · AIOps · 프론트엔드 전체 구축

LLM-as-a-Judge + Pairwise로 모델 교체를 데이터로 결정하고, Prometheus-Loki-Claude를 묶어 AIOps를 구성했습니다.
나이가 그 평가·운영 결과를 다루는 관리자 콘솔을 React-TS로 직접 구현 — 백엔드 API를 화면이 필요로 하는 형태로 설계하고, 사용자 화면(시세·매매·챗봇)까지 프론트엔드 전체를 맡았습니다.

2.38 → 4.04

faithfulness 향상 · 모델 교체

64.3% → 8.3%

환각률 감소

프론트엔드

사용자 + 관리자 화면 구축 주도

74% ↓

캐시 히트 응답 단축 · LLM 호출 스킵

Spring AI 1.0 · pgvector · LLM-as-a-Judge · React · TypeScript · Tailwind · Zustand · Prometheus · Loki · Kafka · Redis

담당: assistant · notification · monitoring + 프론트엔드 (React-TS) 구축 주도

라이브 데모 · frontend · backend

상세 보기→

CAPABILITY · AI 워크플로우

상시

AI에게 일임하지 않고 통제한다 — 하네스 엔지니어링

harness engineering · 규칙 주입 · 가드레일 · 컨텍스트 설계

개발 사이클 단계마다 도구를 나눠 쓰되 설계·결정과 마지막 판단은 항상 사람이 합니다.
코딩 규칙·아키텍처 패턴을 SSOT 문서(CLAUDE.md·컨벤션)로 못박아 AI가 규칙을 벗어나지 못하게 하고, 작업이 끝나면 /revref/apitest가 자동으로 검토·검증해 AI의 실수를 걸러냅니다. AI에게 개발을 맡기는 게 아니라, AI를 관리·감독하는 워크플로우를 직접 엔지니어링합니다.

단계별 분리

분석 · 구현 · 리뷰 도구 분리

자동 검토

/revref · CodeRabbit 컨벤션 점검

규칙 흡수

반복 지적 → 규칙 문서 → 재발 차단

Claude Code · 서버에이전트 · CLAUDE.md · 컨벤션 · /revref · /apitest · CodeRabbit

원칙: AI에게 개발을 일임하지 않는다 — 통제하고 감독한다

상세 보기→

SKILLS

기술 스택

실무 경험 (2년)		학습 · 프로젝트	
LANGUAGE	Java 8 · JavaScript	Java 17 · TypeScript	
BACKEND FW	Spring 4.x (Legacy)	Spring Boot 3.5 · Spring Cloud (Gateway · Eureka · Config · OpenFeign) · Spring AI 1.0 · Spring Security	
FRONTEND	JSP · AngularJS · jQuery — 비동기 렌더링 · 스크롤·이미지 동기화	React · TypeScript · Tailwind CSS · Zustand · Recharts (ants-camp 화면 전체 구축)	
DATA ACCESS	MyBatis · Stored Procedure (T-SQL)	Spring Data JPA · QueryDSL	
DATABASE	MSSQL · MySQL · PostgreSQL	PostgreSQL (+PostGIS · pgvector) · Redis	
ARCHITECTURE	모놀리식 (대규모 운영)	MSA · DDD · Hexagonal · Event-Driven	
MESSAGING / RESILIENCE	—	Apache Kafka · Outbox Pattern · Resilience4j (Circuit Breaker · Retry)	
OBSERVABILITY	운영 로그 분석 · CS 패턴 추적	Prometheus · Grafana · Loki · Promtail · Alertmanager · Zipkin	
AI / LLM	—	Spring AI · RAG · pgvector · LLM-as-a-Judge (Eval · Pairwise) · LLM 기반 AIOps · Human-in-the-Loop	
OPTIMIZATION / PARSING	Apache POI (대용량 문서 파싱 · DRM 시그널 식별)	Google OR-Tools (VRPTW)	
TEST	JUnit · Mockito (팀 도입 주도)	JUnit · Mockito	
REFACTORING / CODE QUALITY	SonarQube (월간 코드리뷰·리팩토링 운영)	—	
DEVOPS	—	Docker · Docker Compose · GitHub Actions	
CLOUD / DEPLOY	—	GCP (Compute Engine · Cloud SQL · Container Registry) · AWS	

이슈를 끝까지 추적하는 운영 오너십

회원제 특허 SaaS의 6개 검색 사이트 유지보수·개선 담당으로, 대량 문서 뷰어·엑셀 다운로드·비교보기를 메인으로 담당했습니다. 운영하면서 마주친 코드·기획·데이터의 불일치를 그냥 넘어가지 않고 원인까지 추적해 해결하는 방식으로 일했습니다. 담당 기능에 머무르지 않고 서비스의 상태 자체를 책임진다는 태도로 운영을 대했습니다.

"이슈를 발견하면 넘어가지 않고, 보고하고, 해결까지 책임졌다."

대량 데이터를 다루는 회원제 SaaS — 6개 검색 사이트의 유지보수·개선, 대량 문서 뷰어·엑셀 다운로드·비교보기를 메인으로, 그 외 사용자 편의성·로그 분석·외부 API 연동·DB 마이그레이션까지 담당 영역을 좁게 한정하지 않고 서비스 전체의 상태를 능동적으로 관리했습니다.

· 플스택 개발자 · 2022.11 ~ 2025.03



이슈를 그냥 넘기지 않는다

코드 동작이 예상하거나 데이터가 이상하면 원인을 직접 추적, 표면 증상에서 멈추지 않고 근본 원인까지 들고 가서 해결.



기획 단계부터 함께 협업한다

기획 자료 · 사용자 관점 의견 제시 · 코드 불일치 검토까지 함께 맡음. 기획팀과 활발히 소통하며 더 나은 방향을 같이 찾음.



운영을 내 것으로 본다

CS 문의·로그·이상 패턴을 능동적으로 모니터링하며 서비스 전체의 상태를 지켜봄.

이슈를 발견·추적·해결한 방식

담당 기능의 경계를 좁게 두지 않았습니다. 운영 중 이상한 점 → 보고 → 원인 추적 → 해결을 능동적으로 반복했고 특히 기획과 코드의 불일치를 발견하면 기획팀과 직접 의논해 정당한 경험이 많았습니다.

CASE #1

기획 단계부터 함께 한 협업 — 자문 · 사용자 관점 · 코드 불일치 검토

기획과 사용자 사이를 잇는 역할로 일하기

기획 자료	기획 초기 단계에서 시스템적으로 가능한 범위 · 비용 · 우회 방법 자문을 요청받음. 받은 기획을 그대로 구현하기보다, 이 방향이 운영에 어떤 부담을 줄지를 함께 검토.
사용자 관점	실제 사용자가 되어 서비스를 써보며 불편한 지점을 직접 체감. 기획에 적힌 흐름과 실제 사용 경험 사이의 간극을 발견하면 의견을 정리해서 공유. "사용자 입장에서 편리한가"를 기준으로 기획팀과 활발히 소통.
코드 불일치 검토	새 기획안이 나오면 반드시 "이 구조가 지금 코드와 맞나?"를 먼저 확인. 기획팀이 알고 있는 시스템 모습과 실제 운영 중인 코드의 동작이 다른 경우를 정리해서 공유 — 어느 쪽을 정정할지 의논해서 결정.
결과	"마상 만들고 보니 의도와 다르다"는 재작업 비용을 사전에 줄이고 기획팀도 시스템의 실제 모습을 더 정확히 알게 됨. 기획·사용자·코드 사이를 잇는 역할도 함께 맡았습니다.

CASE #2

반복 CS 문의의 원인 분석 → 팀 논의 → 사전 차단 구현

문제의 단서를 제공하고 결정된 방향을 책임지고 완수한 사례

상황	"문서 업로드 시 오류가 발생한다"는 사용자 문의가 반복 접수. 1차 대응은 CS팀이 "E-DRM 해제 후 재업로드"를 수동 안내하는 방식이었음.
팀 결정·할당	"수동 안내 대신 시스템 레벨에서 사전 차단하자"는 방향이 팀에서 결정됨. 해당 작업이 본인에게 할당됨.
원인 분석	반복되는 Exception 로그를 분석해, 사내 E-DRM 암호화 문서를 업로드할 때 Java 기반 문서 파서들이 던지는 특정 Exception 패턴이 존재함을 식별. E-DRM 암호화로 문서의 바이너리 구조 자체가 변형되어 파서가 정상 문서로 인식하지 못하기 때문이라, 무작위 에러가 아니라 일정한 원인이 있는 케이스임을 확인.
구현·결과	업로드 단계에서 외부 분석 API에 넘기기 전에 사전 파싱으로 시그널을 감지해 "암호화 해제 후 재업로드" 안내 메시지를 노출. 파일 형식별로 도구를 분리해 MS Office 계열은 Apache POI, HWP는 hwpplib으로 동일 패턴 적용. 동일한 파싱 오류와 E-DRM 케이스를 명확히 분리 → 불명확한 Exception 화면이 명확한 안내로 바뀌고 동일 원인의 CS 문의 재발 없음.

DEEP DIVE — VIEWER UX

이지뷰어 ↔ 도면 팝업 실시간 동기화 — 연간 사용자 만족도 ★2위

메인 담당 기능의 핵심 사용성 작업. 단계별로 문제 원인을 짚어 개선한 결과, 그 해 전체 개선 기능 중 사용자 만족도 2위로 선정

P · 메인 창에는 다수의 특허 문헌을 빠르게 탐색할 수 있는 "이지뷰어"가 있고 "도면 보기"를 클릭하면 별도 컨트롤러 URL을 호출해 전체 HTML을 받아오는 팝업 창이 열리는 구조. A문헌 도면을 본 상태에서 이지뷰어로 다른 문헌을 탐색해도 팝업은 A문헌 도면 상태로 고정되어, 사용자가 매번 "도면 보기"를 다시 클릭해야 하는 불편이 있었음.

A · 단계별 개선 과정 보기

클릭해 펼치기

A · 이지뷰어 문헌 이동 시 팝업 도면이 자동 동기화되도록 개선. 각 단계에서 드러난 문제 원인을 정확히 짚어 다음 단계로 이어감.

1차 — 팝업 자동 갱신 (URL 재호출)

B문헌으로 이동 시 도면 조회 컨트롤러 URL을 다시 호출해 팝업 창 전체를 새로 불러오는 방식.

남은 문제 3가지: ① 페이지 전체 재로딩으로 느림 ② 화면이 순간 하얗게 깜빡임 ③ 새 URL 로드 시 브라우저 보안 정책에 의해 포커스가 팝업 창으로 강제 이동 → 이지뷰어에서 빠르게 문헌을 넘기면 흐름이 매번 끊김.

2차 — 부분 렌더링 (동기 AJAX)

깜빡임·포커스 강탈의 근본 원인이 "팝업 URL을 바꿔 페이지 전체를 재로딩"하는 데 있다고 판단. 팝업 HTML을 고정 영역(헤더·플바) + 가변 영역(도면)으로 분할하고 팝업 URL은 그대로 둔 채 가변 영역의 HTML 조각만 동기 AJAX로 받아 innerHTML로 교체.

해결: 깜빡임·포커스 강탈 사라지고 전송량 감소로 속도 개선. 남은 문제: 동기 AJAX 특성상 응답 도착 전까지 메인 스레드가 멈춰 "순간 멈춤"이 인 지됨.

3차 — 비동기 호출 + 콜백 기반 실행 순서 재구성

AJAX를 비동기로 변경, 단순히 동기→비동기 전환만으로는 부족하고 응답 시점에 의존하는 후속 로직(도면 초기화 JS 등)이 응답 완료 후 실행되도록 콜백 기반으로 코드 구조를 재 정비.

해결: 응답 대기 중에도 사용자 입력에 반응 가능해져 멈춤 현상까지 해소. 12차에서 드러난 모든 문제(깜빡임·포커스 강탈·속도·멈춤)가 최종적으로 모두 개선됨.

R · 연간 사용자 만족도 조사에서 그 해 개선 기능 중 2위로 선정. 기능이 동작하는 데 멈추지 않고 사용자 흐름을 끊지 않는 것까지 가져간 점이 실질적 평가 근거.



회고

AJAX 적용 자체보다, 각 단계의 문제 원인을 정확히 짚어 다음 단계로 이어간 점이 중요했음. 페이지 전체 재로딩 → 부분 교체 → 비동기화로의 흐름은 결과적으로 "브라우저가 강제하는 동작을 피하기 위해 점점 더 가벼운 단위로 갱신 범위를 좁혀 나간 과정"이었음.

작업의 의미

브라우저가 강제하는 동작 단위를 좁혀 사용자 흐름을 보존하는 설계. 사용자 경험을 코드 구조 레벨에서 직접 풀어낸 작업.

점진적 응답성 개선(전체 재로딩 → 부분 렌더링 → 비동기 갱신)의 사고 패턴을 이후 대규모 특허 번역(서버 일괄 응답 → 클라이언트 fan-out + 점진 교체)과 부트캠프 프로젝트의 ants-camp 클라이언트 fan-out 구조에 일관되게 적용했음.

DEEP DIVE — DB MIGRATION

PostgreSQL → MSSQL 마이그레이션 후 슬로우 쿼리 개선

거대 단일 쿼리를 임시 테이블 분할로 개선 — 옵티마이저 추정 원리로 설정 가능한 해결

P · 팀 차원에서 PG → MSSQL(구버전, 2016) 전체 이관을 진행. 마이그레이션 후 특정 쿼리가 눈에 띄게 느려진 케이스를 담당해 개선. 특히 주된 원인은, 중복 제거 문자열 집계에서 PG의 string_agg(DISTINCT)가 MSSQL 2016에는 없어 STUFF + FOR XML PATH + DISTINCT 서브쿼리로 우회해야 했고 이 우회 자체가 큰 성능 저하를 유발.

A · 같은 결과를 내는 여러 SQL 형태를 시도하며 단계적으로 접근:

SQL 형태별 비교 시도 ①~④ — 시행착오 과정

클릭해 펼치기

① 호출 경로 식별 — 영향 범위 먼저 파악

슬로우 쿼리가 발생하는 화면·기능을 먼저 식별해 회귀 검증 범위를 가늠.

이유: 쿼리만 보고 고치면 같은 쿼리가 호출되는 다른 화면에서 회귀가 날 수 있음.

② 서브쿼리 ↔ JOIN ↔ UNION 변환 시도

서브쿼리로 켜 부분들을 JOIN으로, JOIN을 UNION으로 바꿔보며 MSSQL에서 어떤 형태가 빠른지 실행 시간 측정으로 비교.

결과: 일부 쿼리는 개선, 일부는 미미.

③ WHERE절 구조 조정

조건 순서·표현 방식을 다양하게 시도.

결과: 미미한 개선, 근본 해결은 안 됨.

④ 임시 테이블로 단계 분할 — 결정적 해결

복잡한 한 방 쿼리를 단계별로 쪼개서 임시 테이블에 넣고 다음 단계에서 사용.

결과: 큰 개선 확인. 이게 결정적이었음.

R · 벤더가 바뀌어도 기존 환경 대비 동일 수준의 응답 성능 유지하며 슬로우 쿼리 개선 완료.



처리 원리 — 왜 임시 테이블 분할이 통했나

MSSQL 옵티마이저는 테이블 통계(하스토그램)로 각 단계의 행 수를 추정해 조인 순서·방식을 고른다. 참여 테이블과 중간 결과가 많아질수록 추정 오차가 단계를 거듭할수록 커져, 잘못된 조인 순서나 비효율적 조인 방식(예: 해시 조인이 적절할데 중첩 루프를 택함)을 고르기 쉽다. 임시 테이블 분할은 중간 결과를 실제 데이터로 물질화(materialize)해 다음 단계 옵티마이저가 정확한 크기·통계 위에서 다시 판단하게 만드는 효과 — 거대한 한 방 쿼리에서 누락되던 추정 오차를 단계 계계마다 짚아낸다.

옵티마이저 확인 방법: ① SSMS 실제 실행 계획(Ctrl+M)으로 연산자별 추정 행 수 vs 실제 행 수의 괴리와 선택된 조인 방식을 확인 ② SET STATISTICS IO ON · SET STATISTICS TIME ON 으로 쿼리 형태별 논리적 읽기 수·CPU/결과 시간/IO를 수치로 비교 ③ 일반이 크게 맞나가면 UPDATE STATISTICS 로 통계 최신화 여부부터 점검.

학습 연결 고리

이 경험에서 SQL 의존성·벤더 종속성의 한계를 체감해 ORM(JPA)을 학습. 영속성 컨텍스트·벤더 독립 데이터 접근 계층 설계를 익혀 부트캠프 프로젝트의 JPA + QueryDSL 설계에 적용.

DEEP DIVE — CONCURRENCY

임계구역을 긴 로직 뒤에서 맨 앞으로 옮긴 비동기 동시성 설계

1만 건 분할 비동기 처리에서 다중 창 동시 요청으로 한도 우회를 막음 + 비동기 모니터링 누락 제거

P · 1만 건 규모 다운로드를 100건씩 분할 비동기 처리하는 회원제 기능에서, 이용량을 모든 처리 완료 후 집계하는 구조 → 사용자가 여러 창에서 동시에 다운로드하면 검사 시점에 한도가 비어있어 제한 우회(race condition). 비동기 특성상 실제 다운로드의 모니터링 알림도 누락.

A · 임계구역의 위치 자체를 재설계 — 우선별로 트래이드오프 비교:

대안 검토 — 동시성 보호 옵션 비교

REJECTED · DB 락

SELECT ... FOR UPDATE

비동기 처리 시(수 초~수십 초) 동안 락 점유 → 처리량 저하 + 다른 정상 요청까지 블로킹.

REDECTED · 분산 락

Redis · ZooKeeper

단일 DB 환경에 오버엔지니어링. 외부 의존성 추가 + 운영 복잡도 상승. 본 문제는 DB 한 곳에서 풀 수 있음.

CHOSEN · 임계구역 재배치

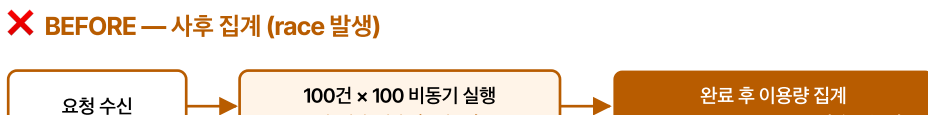
사전 차감 (Front-loading)

이용량 집계를 critical section 가장 앞으로 이동하고 사전 차감 방식 전환. 락 없이도 race 차단 + 실패 시 롤백 가능.

BEFORE / AFTER 구조 비교 보기

클릭해 펼치기

✗ BEFORE — 사후 집계 (race 발생)



❌ 처리 중 새 요청 통과
❌ 만도 우회 가능
❌ 실패 모니터링 누락

✓ AFTER — 사전 차감 (race 차단)



요란: 긴 시간 소모적 이전에 임계구역을 두면 락 없이도 race를 막을 수 있다. 비동기 모니터링은 호출자 예외 컨텍스트 밖에서 발생하므로 완료 분기에서 명시적 메트릭/로깅이 필수.

R · 동시 다발 요청으로 이용량 제한 우회를 구조적으로 차단, 비동기 환경 모니터링 누락 제거.



당시 한계와 이후 학습

당시엔 "여러 방법 중 사전 차감이 맞다"는 판단으로 해결했으나, 락 기반 대안을 체계적으로 비교하지 못한 상태였음. 이후 학습 결과로 정리한 대안: 비관적 락(PESSIMISTIC_WRITE)으로 차감 row를 잠그거나, 낙관적 락(optimistic)으로 충돌 시 재시도하는 방식. 해당 환경에서의 판단 근거: 다운로드 처리 시간이 길어 락 대기 비용이 크므로 사전 차감이 적합. 현재는 환경에 따라 도구를 선택할 수 있는 단계에 도달.

학습 연결 고리

"긴 로직 이전에 임계구역을 둔다"는 동시성 설계 원칙을 체득해 부트캠프에서 분산 락·동시성 제어를 직접 발표(낙관·비관·분산 락 트레이드오프)하고 디2 조부 프로젝트의 seq 제형 race condition에서 의식적으로 비관적 락(PESSIMISTIC_WRITE) 선택.

정합성 보장과 장애 격리를 설계한 B2B 물류 MSA

직전 모놀리식(omin)에서 한 도메인에 결합이 나면 전체가 멈추고 특정 도메인 전용 기술(PostGIS)을 전 팀원이 설치해야 했습니다. 이 비효율을 직접 겪고 **도메인별 독립 배포·기술 선택이 가능한 MSA**로 확장. B2B 도메인을 코드로 녹이기 위해 **DDD 4계층(핵사고 날)**을 적용하고 서비스 간 통신은 **"검증=동기(Feign), 통지=비동기(Kafka)"**로 의도적으로 분리했습니다.

"분산 트랜잭션 없이 정합성을, 장애 전파 없이 통신을."

전국 17개 허브를 거점으로 한 스파르타 물류 — 업체·허브·상품·주문·배송을 MSA로 분리한 B2B 물류 시스템에서 **company-service** 전담 + **DeliveryManager** 도메인 설계. Outbox 패턴 · Circuit Breaker · OR-Tools VRPTW · 비관적 락을 비즈니스 근거에서 출발해 채택.

- 2026 · 부트캠프 5인 · 담당: company-service + delivery-service 네 3개 도메인
- Spring Cloud 2025 · Kafka · OR-Tools · Keycloak

at-least-once

이벤트 정합성 보장

Outbox 패턴 · 분산 트랜잭션 없이

5분 → 3초

배송 배정 자동화

수동 배정 → OR-Tools VRPTW (100건 기준 · 데모)

DB-level

race 막음

비관적 락 · 재시도 로직 없이

스파르타 물류 — MSA 기반 국내 B2B 물류 관리 및 배송 시스템

전국 **17개 지역 허브 센터**를 거점으로 한, **"공급업체 → 허브 → 허브 → 수령업체"** 전 구간을 자동화한 **B2B** 물류 관리 플랫폼.

스파르타 물류는 전국 17개 도/광역시에 허브 센터를 보유한 가상의 B2B 물류 회사입니다. 각 허브는 여러 업체의 물건을 보관하며 주문이 발생하면 시스템은 **출발 허브** → **목적지 허브 간 화물 이동**(허브 배송 담당자)과 **최종 허브** → **수령 업체 배송**(업체 배송 담당자) 두 트랙을 분리해 처리합니다.

예시 시나리오 — "부산 수산물 도매 업체가 경기도 일산의 건조 곡식 가공품 50개를 주문" → 시스템이 **경기도 허브** → **부산 허브**로 화물 이동을 계획하고 도착하면 **부산 허브 소속 업체 담당자**가 최종 업체로 전달.

01 · COMPANY · 담당

업체 관리 (생산 · 수령)

생산업체·수령업체로 구분, 모든 업체는 특정 허브에 소속. 업체 삭제 시 주문/상품/배송 등 단 서비스 이벤트 통지 필요.

02 · HUB · HUBPATH

17개 허브 · 허브 간 경로

전국 도/광역시 17개 센터 관리 + 허브 간 이동 경로 매핑. 모든 화물의 시작/도착점.

03 · PRODUCT

상품 · 재고 관리

모든 상품은 특정 업체와 허브에 소속. 주문 발생 시 입고 감소, 취소 시 복원.

04 · ORDER

주문 라이프사이클

주문 생성 → 재고 검증/저장 → 배송 트리거. 재고 부족 시 주문 실패. 삭제 시 논리 삭제(deleted_at).

05 · DELIVERY · 담당

배송 · 담당자 · 경로 (3개 도메인 통합)

배송 분배 + DeliveryManager(허브당 10명 + 전체 10명) + DeliveryRoute(경로 기록)를 한 컨텍스트로 통합. 매일 **06:00 VRPTW** 자동 재경.

06 · USER · AUTH

4단계 권한 관리

마스터 / 허브 관리자 / 배송 담당자 / 업체 담당자. Keycloak + JWT로 권한 검증.

07 · NOTIFICATION

Slack 메시지 관리

시스템·사용자 발생 슬랙 메시지 전부 엔티티에 저장. Incoming Webhook으로 외부 발송.

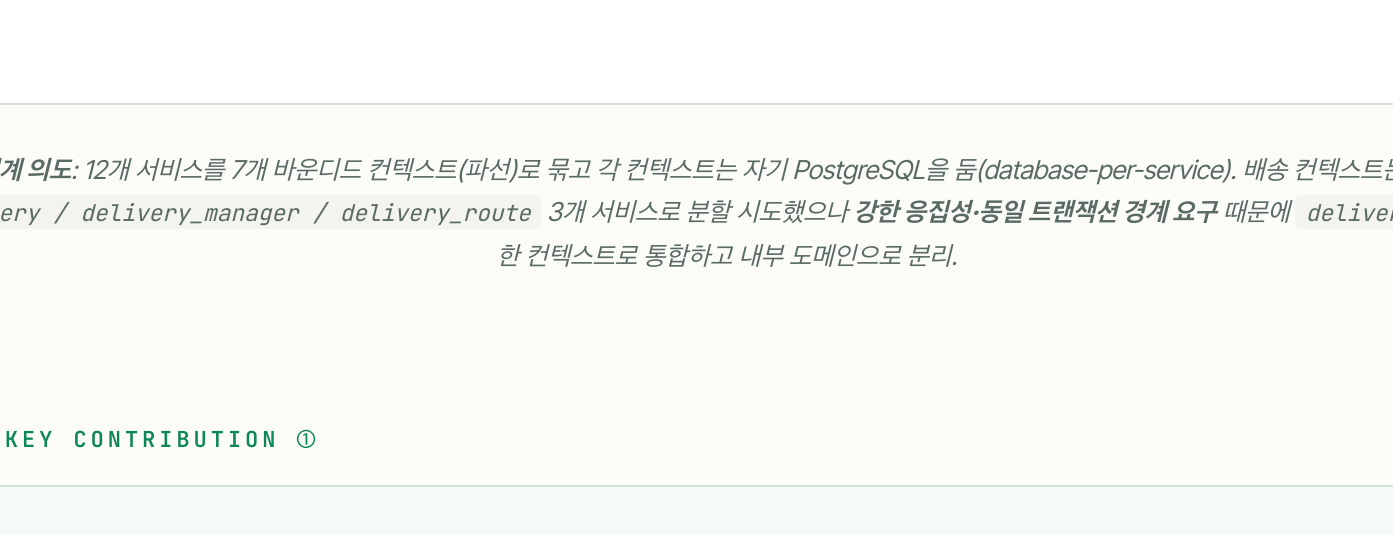
08 · AI

AI 상품 정보 생성

상품 등록 시 LLM 기반 설명 자동 생성. AI 응답 로고는 별도 트랜잭션(REQUIRES_NEW)으로 분리해 본 로직 무량함.

시스템 아키텍처

12개의 마이크로서비스를 **Spring Cloud Gateway · Eureka**로 묶고 통신은 **검증을 동기(Feign)로, 통지를 비동기(Kafka Outbox)로** 분리합니다. 바운디드 컨텍스트별 PostgreSQL 분리(database-per-service)로 서비스 독립성·장애 격리를 확보했습니다.



설계 의도: 12개 서비스를 7개 바운디드 컨텍스트(파선)로 묶고 각 컨텍스트는 자기 PostgreSQL을 둔(database-per-service). 배송 컨텍스트는 처음엔 delivery / delivery_manager / delivery_route 3개 서비스로 분할 시도했으나 **강한 응집성·동일 트랜잭션 경계 요구** 때문에 delivery-service 한 컨텍스트로 통합하고 내부 도메인으로 분리.

CONTRIBUTION #1

이벤트 유실·유일 이벤트 없는 비동기 통지 — Kafka + Outbox 패턴

분산 트랜잭션 없이 **at-least-once** · 결과적 일관성 보장. 단위 테스트로 단일 이벤트 실패가 나머지 발행을 막지 않을 검증

P · 업체 삭제 시 주문·상품 등 타 서비스 통지가 필요한데, "DB 커밋 → Kafka 발행" 사이 발행 실패 시 **이벤트 유실**(DB는 지워졌는데 남들은 모름). 발행 후 DB 롤백 시 유령 이벤트(다들 서비스는 삭제했다고 알고 처리). 분산 트랜잭션(2PC)은 가용성을 해치고 MSA와 부적합.

A · 삭제 트랜잭션 내 OutboxEvent (PENDING) 원자 저장 → DB 커밋과 함께 커밋(또는 함께 롤백). OutboxEventScheduler 주기(Schedule (fixedDelay=10000)) 로 10초마다 PENDING을 Kafka로 발행 → PUBLISHED. 실패 시 PENDING 유지 → 다음 주기 자동 재발행.

R · 분산 트랜잭션 없이 **at-least-once 발행** + **결과적 일관성** 보장. 단위 테스트로 단일 이벤트 실패가 나머지 발행을 막지 않을 검증.

클릭해 펼치기

REJECTED · 2PC

분산 트랜잭션 코디네이터

모든 참여자가 접근을 유지 → 가용성 저하. MSA와 부적합. 코디네이터 자체가 SPOF.

REJECTED · Saga

보상 트랜잭션 체인

다중 단계 트랜잭션 강력하지만 "어떤 묶음을 누구에게 배정"하는 할당(assignment) 문제는 **안 풀**. 지리 공간 시각화/군사 계산용에 가깝고 재택 만족 문제(CSP — 여러 재택을 동시에 만족하는 해를 찾는 문제)에는 부적합.

CHOSEN · OUTBOX

트랜잭셔널 Outbox

도메인 변경과 이벤트를 같은 트랜잭션에 기록 → 별도 publisher가 polling으로 Kafka 발행 → 실패 시 PENDING 유지·재시도. **at-least-once**.

CONTRIBUTION #2

검증은 동기, 통지는 비동기 — Feign + Circuit Breaker로 통신 의도 분리

외부 서비스 장애의 연쇄 전파 차단 — CB(window 10·실패율 50%·OPEN 10s·slow-call 3s) + Retry 3회로 fallback 즉시 반환

P · 외부 서비스 호출이 타임아웃까지 블로킹되면 대기 스레드 누적 → 내 서비스까지 연쇄로 죽음(cascading failure). 단순 동기 호출은 한 서비스 장애가 전체로 번지는 구조.

A · 통신을 의도적으로 분리: **정합성 검증은 동기(Feign)로, 삭제 통지는 비동기(Kafka)로**. user/hub/order 호출은 @CircuitBreaker + @Retry로 감쌌. CB(window 10·실패율 50%·OPEN 10s·slow-call 3s·retry 3회/500ms)로 fallback 즉시 반환.

R · CB OPEN 시 즉시 fallback 반환 → 대기 스레드 누적·연쇄 장애 차단. CB 상태전이(OPEN/CLOSED/HALF_OPEN)·호출차단·slow-call 전체 로깅.

CONTRIBUTION #3

시간창을 제약으로 모델링한 일괄 배송 최적화 — OR-Tools VRPTW

B2B SLA를 좌우하는 시간창 준수율 사후 정렬이 아니라 **솔버의 직접 제약으로 모델링**. 수동 5분 → 자동 3초 (100건 기준 · 데모)

P · B2B 물류는 엄격한 영업시간(예: "오전 9~11시 일고")이 **SLA**의 핵심. 하일터 계약 위반 → **시간창을 제약(constraint — 솔버가 반드시 지켜야 하는 조건)으로 모델링**해야지 후처리 정렬로는 보장 불가. 단순 라운드로빈(들어온 순서대로 담당자에게 균등 배분)은 거리·시간창·차량 부하 균등을 못 고려해 비효율.

A · 매일 06:00 VRPTW를 허브별 당일 배송을 모아 **VRPTW로 일괄 최적화**. 트랙 번호 — 허브 간 = 라운드로빈, 업체 = **OR-Tools VRPTW**. 제약을 모두 만족하는 해가 없으면(infeasible) 시간창을 단계적 완화로 완화해 제시도.

입력 — Haversine 공식(위·경도 좌표로 두 지점 간 직선거리 계산)으로 시간·거리 행렬 생성

제약 — 시간당 공시분 · 라우트 최대 480분(8시간) · 차량당 상한 ceil(통차/담당자수) 로 부하 균등

솔버 — 가까운 지점부터 이어 붙여 초기 경로를 빠르게 만드는 휴리스틱(PATH_CHEAPEST_ARC) + 그 경로를 반복 개선하며 부분 최적에 갇히지 않도록 탐색하는 기법(GUIDED_LOCAL_SEARCH), 탐색 5초 제한

가용성 — 해가 있을 때(feasible) 시간창 단계적 완화 + 스케줄러 레벨 3회 재시도(10초 간격)

R · 시간창을 솔버의 직접 제약으로 모델링해 **SLA 보장 가능한 자동 배정 체계** 확보. 해가 없을 때도 시간창 완화로 가용성 확보.

클릭해 펼치기

REJECTED · K-means

거리 기반 군집화

시간창·차량용량을 제약으로 못 넣음. 클러스터링 후 시간창 검증은 사후 필터링 → 위반 건은 다음 날로 밀려 **SLA 위반**. 초기값에 의존하는 휴리스틱(경험적 어림법)이라 결과도 불안정.

REJECTED · pgRouting

경로 탐색 (A→B 최단)

경로 탐색엔 강하지만 "어떤 묶음을 누구에게 배정"하는 할당(assignment) 문제는 **안 풀**. 지리 공간 시각화/군사 계산용에 가깝고 재택 만족 문제(CSP — 여러 재택을 동시에 만족하는 해를 찾는 문제)에는 부적합.

CHOSEN · OR-TOOLS

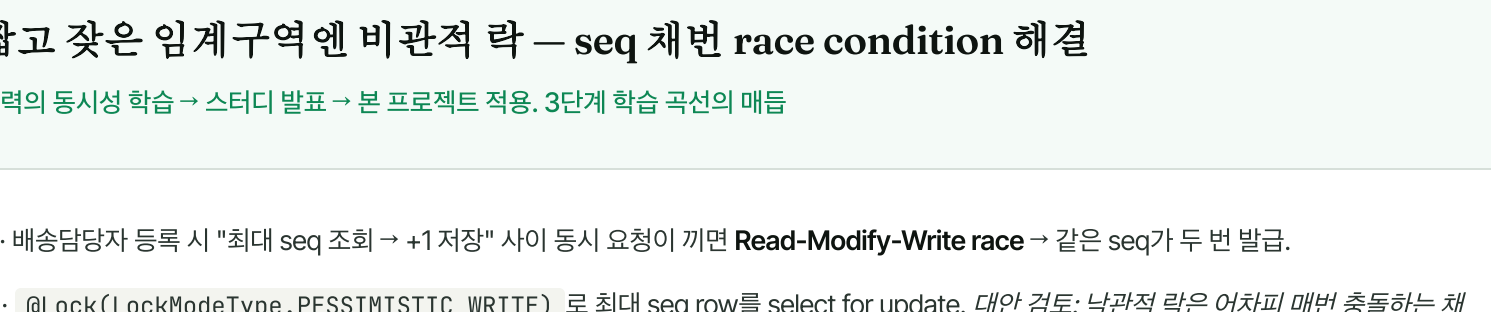
제약 만족 솔버 (CSP)

시간창·차량 용량·적립 시간을 **솔버가 직접 지키는 제약으로 선언** → 솔버가 제약 안에서 최적해 탐색. 해가 없으면 시간창 완화로 단계적 제시도 가능.

왜 일괄(batch)인가 — 실시간 배정 = 들어오는 순서대로 그리다 → **국소 최적**·거리 가까운 사람에게 켜더니 나중 배송이 꼬임. VRPTW는 당일 전체를 한 번에 모아 야간역 최적. B2B는 "죽사"보다 "당일 정시 배송"이 중요 → 일괄이 비즈니스에 더 맞음.

실제 OR-Tools VRPTW 솔버 출력 — 담당자별 경로 시각화

VRPTW · 실속



실제 **VRPTW** 솔버 결과물 지도에 매핑한 경로 시각화. 각 색상이 한 담당자의 경로이며 숫자는 방문 순서.

✓ **5명 담당자별 색상 구분** — 같은 색끼리 묶인 노드가 한 담당자의 하루 동선

✓ 방문 순서 표시(① ② ③ ...) — 솔버가 결정한 실제 순회 경로(sequence)

✓ **허브 출발 → 업체 순회 → 허브 복귀** — VRP의 전형적 라우트 구조가 그대로 출력에 반영

CONTRIBUTION #4

짧고 잦은 임계구역엔 비관적 락 — seq 체인 race condition 해결

경역의 동시성 학습 → 스터디 발표 → 본 프로젝트 적용. 3단계 학습 곡선의 매듭

P · 배송담당자 등록 시 "최대 seq 조회 + +1 저장" 사이 동시 요청이 가면 **Read-Modify-Write race** → 같은 seq가 두 번 발급.

A · @Lock(LockModeType.PESSIMISTIC_WRITE) 로 최대 seq row를 select for update. 대안 검토: 비관적 락은 여차피 매번 충돌하는 체인데 **재시도 비용**도 누적. 분산 락은 단일 인스턴스·토큰로직엔 **과해** → "충돌 찾고 짧은 임계구역"엔 비관적 락이 최적이야 판단.

R · 동시 등록 요청에도 seq 유일성 보장. 재시도 로직 없이 **DB 레벨에서 차단**. 추가로 락 타임아웃 3초를 명시해 장기 트랜잭션·데드락 시 LockTimeoutException 으로 빠른 실패 처리 — 무한 대기로 인한 스레드 점유 차단. 동시 등록 시나리오 단위 테스트로 seq 유일성을 검증했다.

클릭해 펼치기

REJECTED · 낙관적 락

버전 충돌 → row 버전

재번은 매번 같은 **row**를 만짐 → 거의 항상 버전 충돌 발생 → 재시도 비용만 누적되고 처리량 저하.

REJECTED · 분산 락

Redis · ZooKeeper

단일 인스턴스·토큰로직엔 **오버엔지니어링**. 외부 의존성도 높여냄.

CHOSEN · 비관적 락

SELECT ... FOR UPDATE

임계구역이 **짧고 충돌이 잦음** → 락 대기 부임은 직고 충돌 차단 효과는 큼. 재시도 로직 없이 DB 레벨에서 차단.

배포 인프라 — IaC 한 번으로 AWS + Cloudflare 전 구간 프로비저닝

16개 컨테이너 규모의 MSA를 전·사·데모용으로 **월 ~\$14**에 운용하기 위해, 인프라 전체를 **Terraform** 단일 코드베이스(IaC)로 선언하고 **GitHub Actions** + **OIDC**로 빌드·배포를 자동화했습니다. "비용 · 보안 · 운영부담"을 판단 기준으로, 매니지드 오케스트레이션 대신 단일 EC2 + Cloudflare Tunnel 조합을 의도적으로 선택했습니다.

월 ~\$14

비용 최적화

ALB/EIP 제거 · 평일 09~21시만 자동 가동 (24/7 대비 ~64% ↓)

keyless

OIDC 기반 CI/CD

장기 액세스키 0개 · 임시 자격증명으로 9개 이미지 push

1× apply

IaC 단일 적용

VPC·EC2·RDS·ECR·IAM·Scheduler + Cloudflare까지 선언적 관리

CONTRIBUTION #5

Terraform IaC + OIDC 키리스 CI/CD + Cloudflare Tunnel 무포트 노출

매니저드 파우어 대신 "전시 목적에 맞는 최소 비용 · 최소 공격면" 아키텍처를 직접 설계·구축

CI · GitHub Actions — develop push 시 9개 서비스를 **matrix 병렬 빌드** — ECR push(:latest , :sha) . **OIDC**로 AWS 임시 자격증명을 발급받아 장기 액세스키를 저장하지 않음. type=gha · 리미트 캐시로 빌드 시간 단축.

IaC · Terraform — VPC(NAT 없는 퍼블릭 서브넷)·EC2·RDS(프리티어)·ECR·IAM(OIDC role)·EventBridge Scheduler·Lambda를 하나의 코드로 선언. **Cloudflare(Tunnel·DNS·SSL)까지 같은 apply** 로 생성하고 터널 토큰을 EC2 user_data에 자동 주입.

네트워킹 · **Cloudflare** — ALB/EIP 없이 **Tunnel로 오리진을 노출**해 인터넷 포트를 전부 닫고(공격면 ↓), 무료 Universal SSL로 HTTPS 종단. 운영시간 ·외부발행 **Worker가 점검 페이지로 폴백**.

비용 중앙 · **EventBridge + Lambda** — 평일 **08:40 start / 21:00 stop**으로 EC2·RDS를 자동 전원 제어, 전시 시간대만 가동해 비용을 절반 이하로 절감.

AI가 일하는 방식을 설계하는 것 — 하네스 엔지니어링

개발을 AI에게 일임하는 것이 아니라, **AI가 이 코드베이스 규칙을 벗어나지 못하도록 통제·감독하는 체계**를 설계합니다. AI의 결과물은 빠르고 그럴듯하지만 맥락을 모른 채 자주 틀리기에, antcamp (가상 주식 거래 MSA · 도메인 서비스 7개)에 규칙 문서(SSOT) · 자동 검토/테스트 스킴 · 가드레일을 돌려 **AI의 출력은 사람의 검증을 통과해야만 코드베이스에 반영되도록** 했습니다. 코드의 최종 판단과 책임은 항상 사람이 집니다.

harness engineering — AI를 통제·감독하는 체계를 엔지니어링

반복 타이핑만 시로 처리하되, 무엇을-언제 참조할지, 무엇을 실행하면 안 되는지, 무엇을 반드시 검증할지를 사람이 못박습니다. AI가 단독으로 코드를 확정하는 일은 없습니다 — 모든 출력은 규칙·자동 검토·빌드 검증을 거쳐 **사람이 최종 확인**합니다. 도구(모델·CLI)는 빠르게 바뀌지만, 규칙 문서·검토 스킴·가드레일이라는 **관리 체계**는 프로젝트를 넘어 남습니다.

• SSOT 규칙 문서 3종 • 자동 검토·검증 A · B · C • 합격 판정은 사람이

규칙을 단일 출처(SSOT) 문서로 주입한다

AI가 일관되게 일하려면 "무엇이 규칙인가"가 **한 곳에** 있어야 합니다. 세 문서가 서로를 참조하며 구조 · 규칙 · 절차를 나눠 가지고, C리뷰 봇(.coderabbit.yaml)과 동일한 기준을 공유합니다.

구조 · 컨벤션

CLAUDE.md

핵사고날 4레이어, 패키지·네이밍 표, 게이트웨이 인증 흐름(X-User-Id 신뢰 경계), Config Server 외부화, common 모듈 규약.

코딩 규칙

coding-conventions.md

트랜잭션 범위·JPA 캡슐화·DDD/레이어 경계·MSA 경계·record VO·동시성/분산락·로깅 보안·common 재사용. 위반은 심각도로 분류.

절차

automation-workflow.md

코드 작업 후 자동 검토(A) · 빌드/테스트/API(B) · Slack 보고(C) 워크플로우의 트리거와 흐름을 정의. "언제 무엇을 자동으로 할지"를 명문화.

작업 단계에 맞춰 자동으로 도는 워크플로우 A·B·C

규칙은 "지켜야 한다"로 끝나지 않게, 작업이 끝나면 자동으로 검토·검증·보고가 돌도록 슬래시 스킴과 스크립트로 묶었습니다.

워크플로우	트리거	수단
A · 코드 작업 후 자동 검토	새 파일/메서드 추가, 50줄+ 수정, common/ 변경	/revref 스킴
B · 빌드→테스트→API 실행	런타임 동작에 영향 주는 변경 완료	/apitest 스킴
C · 완료 후 Slack 보고	A/B 또는 주요 작업 완료	notify-slack.sh

A. /revref — 컨벤션 준수 자동 검토

git diff 변경분(--context)이나 지정 파일을 대상으로, **차원별 리뷰 에이전트를 병렬 실행**해 위반을 심각도와 근거(conventions \$N)와 함께 보고합니다. 같은 일반론 지적이 반복되면 규칙 문서로 흡수합니다.

/revref 실행 결과 — 차원별 병렬 검토 종합

WORKFLOW A

• revref 결과 — notification-service 알림 상태별 카운트 API (8개 파일)

CLICK TO ZOOM

지적 사항

심각도 : 🔴
위치 : -
문제 : 없음 (마지막 차단 사유 없음)
제한 : -

심각도 : 🟡
위치 : NotificationStatusCountResponse.java:24
문제 : total을 4개 상태 하드코딩 합산 (pending+sent+failed+actionFailed). 현재는 정확하나 AlertStatus 값 추가 시 total에서 조용히 누락 (3개 에이전트 독립 지적)
제한 : counts.values().stream().mapToLong(Long::longValue).sum()로 맵 전체 합산

심각도 : 🟡
위치 : NotificationStatsQuery.java:14-17
문제 : 8-인자 NotificationSearchCriteria를 위치 인자 + null 리터럴 6개로 생성 → 필드 변경 시 취약 (타입 달라 오버처는 컴파일 예외로 걸림)
제한 : criteria.forStats(severity, source) 정적 메소드 (선택)

심각도 : 🟡
위치 : NotificationController.java:93
문제 : {범위 밖·선재 미수} 기존 receiveSlackAction이 Slack payload 전문 로깅
제한 : notificationId 등 비민감 식별자만 로깅, 별건 처리

변경분을 차원별 에이전트가 병렬 검토해 🔴/🟡 심각도 · 파일:라인 · 근거(conventions \$N)와 함께 보고한다.

B. /apitest — 빌드부터 게이트웨이 시나리오까지

1-2 · 빌드 · 단위

gradlew

build -x test → test 실패 시 컴파일/테스트 원인을 요약하고 여기서 중단, 결과 저장 후 실패 보고.

3-4 · 인프라 · 시나리오

docker *.http

게이트웨이 :8080/actuator/health 폴링 후 scenario*.http 를 게이트웨이 기준으로 실행, 단계별 상태코드·본문으로 통과/실패 판정.

5 · 보존

tests/results

타입스탬프 폴더에 summary.md 표·요청·응답 저장. 토론·시크릿은 마스킹, 결과물은 .gitignore .

/apitest 실행 결과 — 빌드→테스트→게이트웨이 시나리오

WORKFLOW B

apitest 결과 — notification-service 상태별 카운트 API

CLICK TO ZOOM

1) 빌드 :apps:notification-service:build -x test → SUCCESSFUL

2) 단위 테스트 :apps:notification-service:test → NO-SOURCE (해당 서비스 테스트 소스 없음)

3) 인프라 Postgres(pgvector) + Redis 컨테이너 + 로컬 bootJar(:8098) 경찰 기동 → 실패 DB(p_slack_messages)에 8인 시드, 게이트웨이 계약대로 X-User-Id/X-Role 헤더 직접 주입.

4) API 실행 — 8/8

#	케이스	엔드포인트	기대	실제	결과
01	필터 없음	/admin/stats	8/2/4/1/1	8/2/4/1/1	✓
02	severity=CRITICAL	...?severity=CRITICAL	3/1/1/1/0	3/1/1/1/0	✓
03	source=PROMETHEUS	...?source=PROMETHEUS	4/1/3/0/0	4/1/3/0/0	✓
04	두 필터 동시	...?severity=CRITICAL&source=PROMETHEUS	2/1/1/0/0	2/1/1/0/0	✓
05	결과 0인	...?severity=INFO&source=GRAFANA	0/0/0/0/0	0/0/0/0/0	✓
06	잘못된 enum	...?severity=BOGUS	400 INVALID_INPUT	400 INVALID_INPUT	✓
07	목록 일관성 (전체)	/admin?page=0	totalElements=8	8	✓
08	목록 일관성 (CRITICAL)	/admin?severity=CRITICAL	totalElements=3	3	✓

빌드·단위 테스트 후 게이트웨이 헬스 폴링 → 시나리오(http)를 단계별 상태코드·본문으로 통과/실패 판정한다.

C. notify-slack.sh — 작업을 깨지 않는 보고

SCRIPTS/NOTIFY-SLACK.SH

```
# 성공/실패 + 제목 + 본문(또는 summary 파일)으로 Incoming Webhook 보고
scripts/notify-slack.sh --status success \
  --title "apitest: trade-service" \
  --file "tests/results/20260628_153000/summary.md"

# 실패 미설정 시 → 경고만 출력하고 정상 종료 (빌드/작업을 깨지 않음)
```

외부 전송이므로 처음에는 보고 대상·내용을 사용자에게 확인받고, 사소한 변경(문서/오터)은 보고하지 않습니다.

notify-slack.sh 보고 — 작업 완료 Slack 알림

WORKFLOW C

☆ # dev-report

메시지 캔버스 추가 +

dev-report

apitest: notification-service 상태별 카운트 API

apitest: notification-service — 알림 상태별 카운트 API

- 대상: GET /api/notifications/admin/stats (신규) + 회귀: GET /api/notifications/admin

- 일시: 2026-06-29 00:56 (KST)

- 변경 파일: NotificationController, NotificationStatsRequest(신규), NotificationStatsQuery(신규), NotificationStatusCountResponse(신규), NotificationQueryService, NotificationRepository(port), NotificationRepositoryImpl, NotificationQueryRepository(GROUP BY 집계)

성공/실패 · 제목 · 요약은 Incoming Webhook으로 전송한다. 실패 미설정 혹은 실패 전송 오류 시 경고만 출력하고 작업을 깨지 않는다.

가드레일과 컨텍스트를 설계한다

생성을 빠르게 하되, 위험한 자동화는 막고(가드레일) AI가 틀린 맥락으로 추측하지 않게(컨텍스트) 합니다.

가드레일 — 무엇을 못 하게 하나

- 합격 판정은 사람이 — 검토·테스트는 자동화하되 통과 여부는 직접 결정, 🔴 항목은 반드시 수정 후 빌드로 재검증
- 외부 전송은 확인 후 — Slack 보고는 처음에 사용자 승인
- 시크릿·PII 마스킹 — 테스트 결과·로그에 JWT/ KIS_* / *_API_KEY /비밀번호 평문 금지(\$8)
- 테스트 산출물(tests/results/)은 커밋 대상 아님 (.gitignore)
- 신뢰 경계 — 도메인 서비스는 JWT를 직접 검증하지 않고 게이트웨이가 주입한 X-User-Id 만 신뢰

컨텍스트 — 무엇을 언제 보게 하나

- SSOT 상호참조 — 세 문서가 서로를 가리켜 중복·표류 방지
- 설정 외부화 인지 — "이 레포에서 application.yaml 찾기 말 것", 실제 설정은 Config 레포(dev)
- 작업 유형별 참고문서 라우팅 — RAG 작업 → rag-architecture.md , 알림 작업 → aiops-architecture.md
- 취발성 큰 정보(스키마·설정)는 문서에 박제하지 않고 그때그때 실출

왜 이렇게까지 하는가

AI 생성물은 빠르지만 이 코드베이스의 맥락을 모른 채 그럴듯하지만 틀린 코드를 자신 있게 내놓습니다. 그 실수가 코드베이스에 들어오지 못하도록 규칙(SSOT)·자동 검토·검증·가드레일로 여러 겹의 관문을 두고, 사람은 생성이 아니라 검증과 최종 판단에 시간을 씁니다. AI는 빨라지되 사람의 통제 밖에서 코드를 확정하지 못합니다. 이 관리 체계는 모델·도구가 바뀌어도 남습니다.

AI에게 개발을 일임하지 않는다 — 통제하고 감독한다.